MediQ

Intelligent Symptom Diagnostics

MediQ

An End-to-End Technical Report

A machine-learning system for symptom-based disease prediction and clinical-style guidance, from raw data to deployed web app.

Author	Reebal
Document	Technical / portfolio project report
System	Random Forest classifier + Flask web application
Stack	Python, scikit-learn, pandas, Flask
Scope	41 diseases · 132 symptoms · 4,920 training records
Date	30 June 2026

Abstract

MediQ is an end-to-end machine-learning system that maps a set of patient-reported symptoms to a probable disease and surrounds that prediction with practical, human-readable guidance — a description of the condition, recommended precautions, dietary advice, a medication overview, and lifestyle suggestions. Under the hood, a **Random Forest classifier** is trained on 4,920 labelled records spanning **41 diseases** described by **132 binary symptom features**. Around the model sits a curated **knowledge base** assembled from seven reference tables, and a lightweight **Flask web application** that exposes the model through a polished diagnostic dashboard with autocomplete, a confidence gauge, and a ranked list of differential diagnoses. This report walks through the entire project: the motivation, the data, the engineering that turns messy spreadsheets into a clean feature matrix, the mathematics of the learning method, the evaluation and its important caveats, the application architecture, and an honest discussion of limitations and next steps.

41	132	4,920	300	80 / 20
diseases	symptoms	training records	trees in forest	train / test split

1 Introduction & Motivation

Most people, when they feel unwell, do the same thing: they type their symptoms into a search engine and try to make sense of a wall of conflicting results. That experience is anxious, noisy, and rarely structured. MediQ began from a simple question — *what if a model could take a clean list of symptoms and respond the way an organised triage assistant would?* Not to replace a doctor, but to demonstrate how a structured dataset and a well-understood learning algorithm can turn scattered symptom information into a single, ranked, explainable answer.

The motivation is therefore twofold. First, it is a **practical data-science exercise**: the full lifecycle of a supervised-learning product is on display here — sourcing data, cleaning it, engineering features, training and evaluating a model, persisting artifacts, and serving them behind an API with a real user interface. Second, it is a study in **responsible framing**. Health is exactly the kind of domain where a model that looks impressive can be dangerously misleading, so a recurring theme of this report is being candid about what the system genuinely does and does not establish.

The name **MediQ** captures the intent: a *medical IQ* layer that reasons over symptoms, paired with a clean interface that presents its reasoning rather than hiding it.

2 Aim & Objectives

The aim of the project is to design, build, and deploy an interpretable machine-learning pipeline that predicts a likely disease from a patient's selected symptoms and presents supporting, human-readable health guidance through a web interface.

Concrete objectives

- **Data integration.** Consolidate eight separate CSV sources into one coherent training matrix and one structured knowledge base.

- **Robust preprocessing.** Normalise inconsistent disease and symptom labels, repair known typos, de-duplicate columns, and encode symptoms as machine-readable features.
- **Model training.** Train a multi-class classifier that outputs not just a label but a calibrated-style probability for every candidate disease.
- **Explainability.** Surface the top-five differential diagnoses with confidence scores, plus a transparent symptom-severity score, so the output is never a single opaque guess.
- **Deployment.** Wrap the trained model in a Flask service with a responsive dashboard, autocomplete search, and a clear safety disclaimer.
- **Reproducibility.** Make the whole pipeline re-runnable from a single training script that regenerates every artifact deterministically.

3 System Overview

MediQ is organised as two cooperating stages connected by a small set of on-disk artifacts. The **offline training stage** (`train_model.py`) reads the raw data, builds the feature matrix and knowledge base, fits the model, and writes everything to an `artifacts/` folder. The **online serving stage** (`app.py`) loads those artifacts once at start-up and answers prediction requests. This separation means the expensive work happens once, and the web app stays fast and stateless.

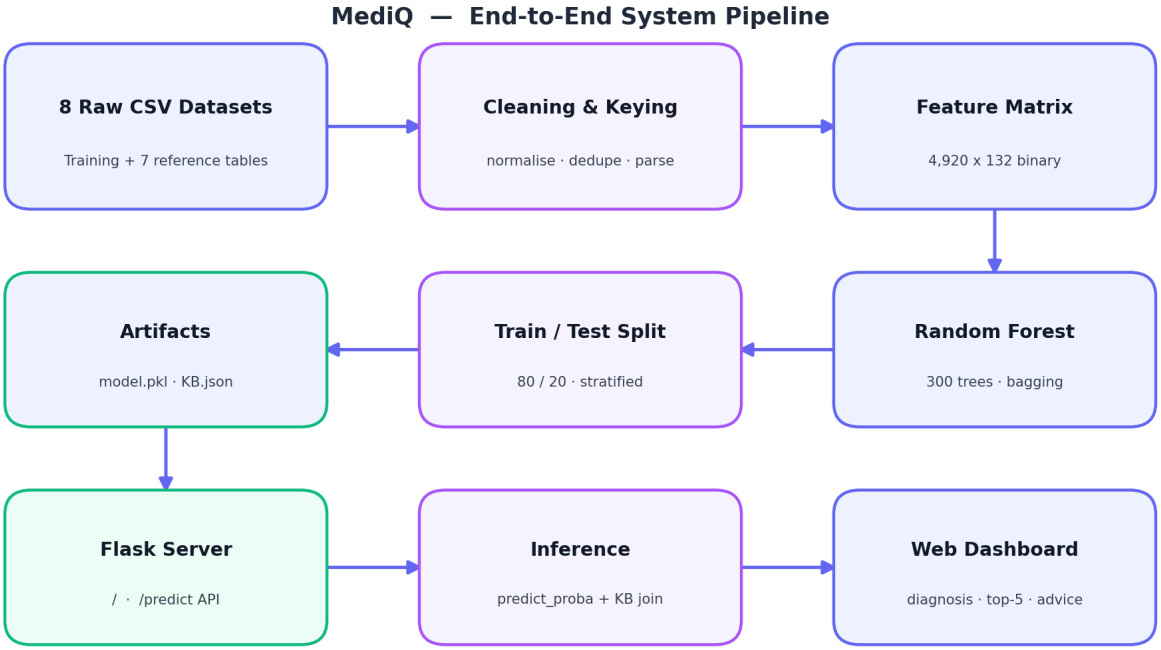

Figure 1 — The MediQ pipeline, from eight raw CSVs through cleaning, feature construction, model training, and persisted artifacts, to the Flask server, inference, and web dashboard.

The four artifacts that bridge the two stages are the trained model bundle (`disease_model.pkl`), the symptom lexicon used to power the UI autocomplete (`symptoms.json`), the disease knowledge base (`knowledge_base.json`), and a plain-text evaluation report (`model_report.txt`).

4 The Data

MediQ is a data-fusion project as much as a modelling one. Eight CSV files supply two different kinds of information: the **signal** the model learns from, and the **reference content** that makes a prediction useful to a human reader.

4.1 The eight source files

File	Shape	Role
Training.csv	4,920 × 133	Labelled training data: 132 binary symptom columns + prognosis label
Symptom-severity.csv	133 × 2	Clinical severity weight (1–7) for each symptom
symtoms_df.csv	4,920 × 6	Representative symptoms listed per disease
description.csv	41 × 2	One-line description of each disease
precautions_df.csv	41 × 6	Up to four precautions per disease
medications.csv	41 × 2	Typical medications per disease (list)
diets.csv	41 × 2	Recommended dietary items per disease (list)
workout_df.csv	410 × 4	Lifestyle / workout recommendations per disease

Only `Training.csv` drives the classifier. The remaining seven tables are *reference* data: they never touch the model's decision boundary but are joined to its output at inference time to produce the rich result cards the user actually sees.

4.2 Composition and class balance

The training set is unusually tidy. Every one of the 41 diseases is represented by exactly **120 records**, giving a perfectly balanced 4,920-row matrix. Each record is a row of 132 zeros and ones indicating which symptoms are present, followed by the disease label. The severity lexicon assigns each symptom a weight from 1 (mild, e.g. *itching*) to 7 (severe, e.g. stroke-like signs), which MediQ later aggregates into a symptom-burden score.

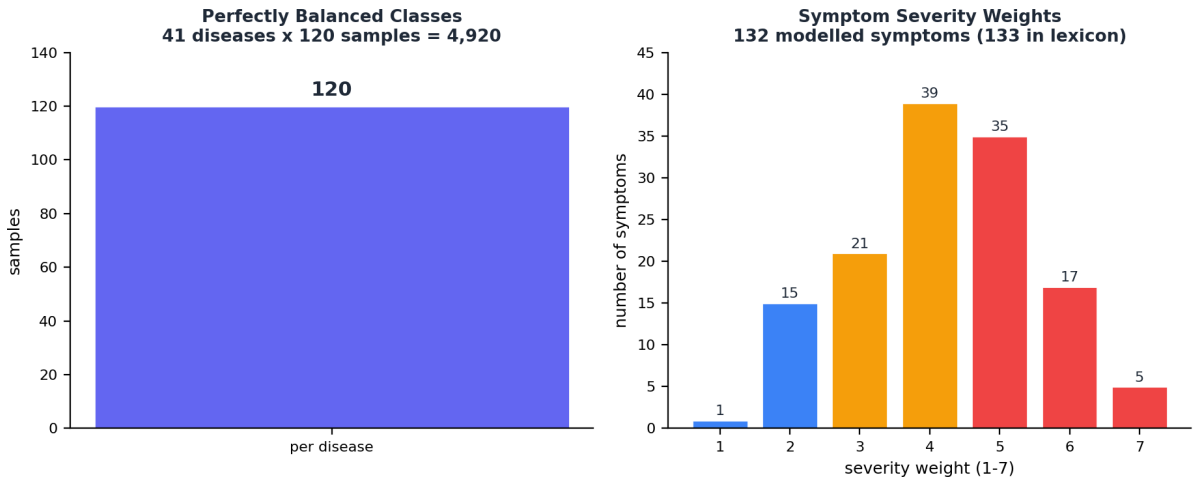

Figure 2 — Left: the dataset is perfectly class-balanced at 120 samples per disease. Right: distribution of the 1–7 severity weights across the symptom lexicon, coloured by mild (blue), moderate (amber), and severe (red).

This balance is a double-edged sword. It removes any need for resampling and makes accuracy a meaningful headline metric, but — as Section 8 discusses — it also hints that the data is *synthetically*

templated rather than drawn from real, noisy patient encounters.

5 Data Engineering & Preprocessing

Raw medical spreadsheets are messy: trailing spaces, inconsistent capitalisation, duplicated columns, embedded typos, and list-valued cells stored as strings. The training script invests heavily in turning this into something a model can trust. Four ideas do most of the work.

5.1 Normalisation and key harmonisation

Two helper functions act as the project's spell-checkers. A disease-key function lower-cases names, collapses whitespace, and repairs known dataset typos — `diseae` → `disease`, `paroymsal` → `paroxysmal`, `hemmorhoids` → `hemorrhoids` — so that the same condition spelled three different ways across three files still resolves to one canonical key. A symptom-key function snaps every symptom to `snake_case`, so `'spotting urination'` and `'spotting_ urination'` become a single feature. This shared keying is what lets eight independently-authored files be joined without silent mismatches.

5.2 Column de-duplication

Some symptom columns appear more than once in the raw training file. Rather than dropping data, MediQ merges duplicate columns by taking the element-wise maximum of their binary values — if *any* copy records the symptom as present, the merged column records it as present. This preserves signal while guaranteeing one clean column per symptom.

5.3 Feature encoding

After cleaning, each patient record is represented as a **132-dimensional binary vector**. Position *j* is 1 if symptom *j* is present and 0 otherwise. This bag-of-symptoms encoding is simple, sparse, and naturally suited to tree-based models, which split on individual feature thresholds.

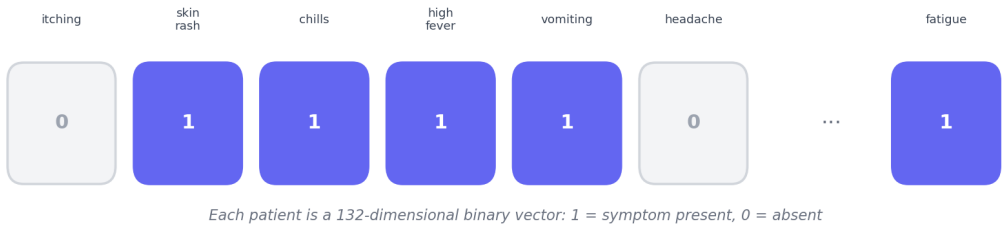

Figure 3 — A patient is encoded as a 132-dimensional binary vector over the symptom vocabulary; the same encoding is reconstructed from the user's selections at inference time.

5.4 Building the knowledge base

The seven reference tables are folded into a single dictionary keyed by canonical disease. For every disease, MediQ collects its description, list-parsed diets and medications, up to four precautions, de-duplicated lifestyle recommendations, and a set of representative symptoms. List-valued cells — stored as Python-literal strings like `"['Antifungal Cream', 'Fluconazole']"` — are safely parsed with a literal evaluator that falls back to delimiter splitting. The result is a knowledge base with **complete coverage**: all 41 diseases carry a description, diets, medications, precautions, and lifestyle advice.

6 Methodology & Mathematics

MediQ frames diagnosis as a **supervised multi-class classification** problem. Given a symptom vector $x \in \{0,1\}^{132}$, the goal is to predict a disease label y from one of 41 classes. The chosen estimator is a **Random Forest** — an ensemble of decision trees — for three reasons: it handles high-dimensional binary features gracefully, it is robust to irrelevant symptoms, and it produces per-class probabilities almost for free.

6.1 Decision trees and the Gini criterion

A single decision tree repeatedly splits the data on the symptom that best separates the diseases. "Best" is measured by the reduction in **Gini impurity**. For a node whose samples fall into classes with proportions p_k, the impurity is:

$$G = 1 - \sum_k p_k^2$$

A pure node (all one disease) has G = 0. Each candidate split is scored by the weighted Gini impurity of its children, and the tree greedily chooses the split that lowers impurity most. Splitting continues until the leaves are pure or no useful split remains — at which point a leaf simply stores the class distribution of the samples that reached it.

6.2 From one tree to a forest: bagging

A lone decision tree is expressive but high-variance: small changes in the data can produce a very different tree. A Random Forest tames this with two layers of randomness. First, **bootstrap aggregating (bagging)** — each of the T = 300 trees is trained on a different bootstrap sample (a random draw, with replacement, of the training rows). Second, the **random subspace** method — at every split a tree may only consider a random subset of features, here $\sqrt{p} \approx \sqrt{132} \approx 11$ symptoms. Together these decorrelate the trees, so their errors tend to cancel when averaged.

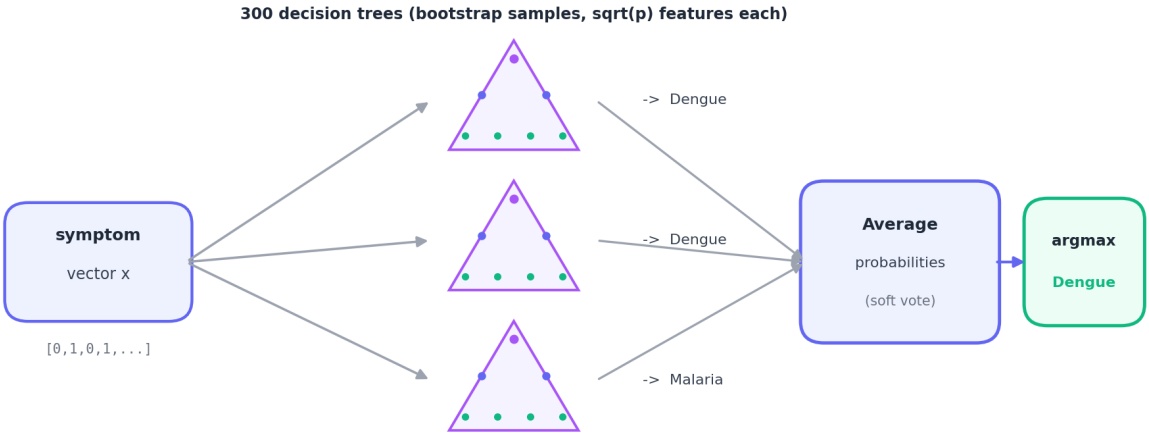

Figure 4 — Each tree sees a different bootstrap sample and a random feature subspace; the forest averages their class probabilities (soft voting) and returns the arg-max disease.

6.3 Soft voting and probability estimates

Rather than letting each tree cast a single hard vote, scikit-learn's Random Forest **averages the class probabilities** the trees produce. If tree t estimates $p_t(c \mid x)$ for class c, the forest's probability is the mean across trees:

$$P(y = c \mid x) \;=\; (1 / T) \sum_{t=1}^{T} p_t(c \mid x)$$

The predicted disease is simply the arg-max, $\hat{y} = \arg\max_c P(y = c \mid x)$. Crucially, the *whole* probability vector is kept — this is what powers MediQ's confidence gauge and its top-five differential-diagnosis list. In code this is the single call `model.predict_proba(X)`, whose output is sorted to rank candidate diseases.

6.4 Label encoding and class weighting

Disease names are mapped to integer indices with a `LabelEncoder` before training and inverted afterwards for display. The forest is also configured with `class_weight = "balanced_subsample"`, which reweights classes inside each bootstrap so that — had the data been imbalanced — rarer diseases would not be drowned out. On this already-balanced dataset it is a harmless safeguard.

6.5 The symptom-burden score

Alongside the probabilistic prediction, MediQ reports a transparent, non-learned **severity score** — the sum of the clinical weights of the selected symptoms:

$$S \;=\; \sum_{s \,\in\, \text{selected}} \text{weight}(s)$$

This deliberately simple heuristic gives the user an at-a-glance sense of how acute their reported symptom set is, independent of the model's disease guess — a higher S flags a heavier symptom burden and nudges the user toward seeking care.

6.6 Training protocol

The data is split **80 / 20** into training and held-out test sets using a fixed random seed (`random_state = 42`) and **stratification** on the disease label, so every class keeps its proportion in both splits. The model is fit only on the training portion; the 984-row test set is never seen during fitting and is used solely for the evaluation in Section 8.

7 Model Configuration

The estimator is scikit-learn's `RandomForestClassifier`, configured as follows:

Hyperparameter	Value	Rationale
n_estimators	300	Many trees for a stable averaged probability
max_features	sqrt	Random subspace ≈ 11 of 132 features per split; decorrelates trees
class_weight	balanced_subsample	Guards against class imbalance per bootstrap
random_state	42	Deterministic, reproducible training
n_jobs	-1	Train trees in parallel across all CPU cores

Hyperparameter	Value	Rationale
test_size	0.20 (stratified)	Held-out evaluation preserving class ratios

Every other parameter is left at its scikit-learn default — notably trees are grown to full depth, which is appropriate given the clean, low-noise feature space.

8 Results & Evaluation

On the 984-record held-out test set the model achieves **100% accuracy** — every disease in the test split is classified correctly. The confusion matrix is a clean diagonal, and precision, recall, and F1-score are all 1.00 for all 41 classes (24 test samples each).

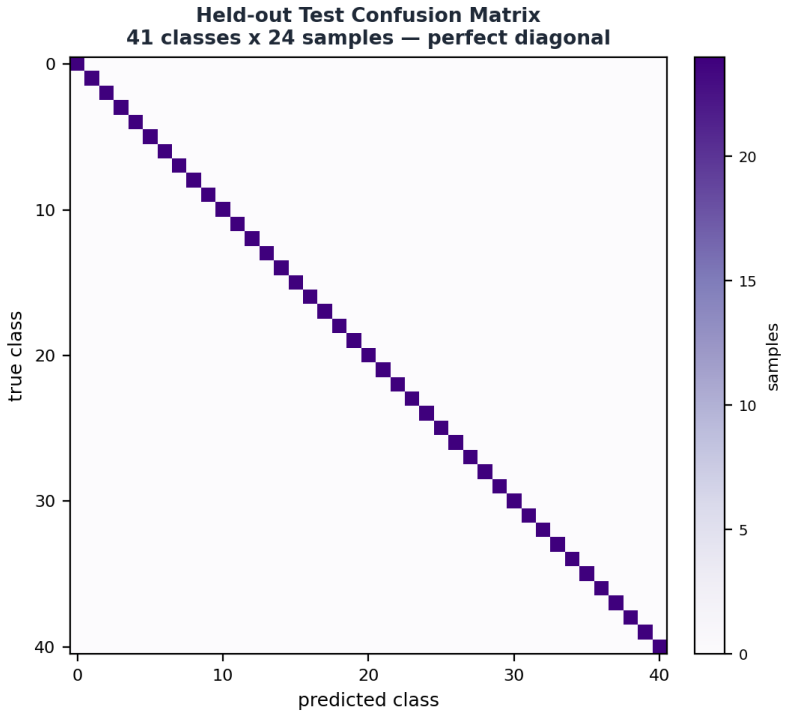

Figure 5 — The held-out confusion matrix is perfectly diagonal: all 41 diseases × 24 test samples are classified correctly.

A perfect score is the kind of result that should invite scrutiny rather than celebration. The honest interpretation is that this number reflects the *nature of the dataset* far more than any clinical capability of the model.

> ■ **Why 100% accuracy is a caveat, not a triumph**
>
> The dataset is synthetically templated — each disease maps to an almost fixed symptom signature, classes are perfectly balanced, and there is little of the ambiguity, overlap, or noise found in real patients. A flexible model like a Random Forest learns these clean patterns trivially, and the test set is drawn from the same distribution. The result demonstrates that the pipeline is correct and the model fits the data; it does **not** demonstrate real-world diagnostic accuracy. On messy clinical data, performance would be materially lower.

What the evaluation *does* validate is end-to-end correctness: the cleaning, keying, encoding, training, and serialisation all work, and the served model reproduces training-time behaviour exactly.

9 The Application

The trained artifacts are served by a compact Flask application that loads the model bundle, symptom lexicon, and knowledge base once at start-up. It exposes two routes: a `/` route that renders the single-page dashboard, and a `/predict` JSON API that accepts a list of symptom IDs and returns a structured result.

9.1 Inference flow

When a request arrives, the server reconstructs the 132-dimensional binary vector from the user's selected symptoms, calls `predict_proba`, ranks the classes, and joins the top result against the knowledge base to attach the description, precautions, diets, medications, lifestyle advice, and common symptoms. It also computes the severity score and packages the top-five diseases with their probabilities for the differential-diagnosis panel.

9.2 The interface

The dark-themed dashboard is built for clarity. A searchable, keyboard-navigable autocomplete lets users add symptoms as colour-coded chips (shaded by severity); a primary action runs the model; and the results render as a hero card with an animated confidence gauge, a prominent **medical safety disclaimer**, six information cards, and a horizontal-bar chart of the top five differential diagnoses. The whole experience is designed to present the model's reasoning transparently rather than reducing it to a single verdict.

10 Limitations & Ethical Considerations

- **Not a medical device.** MediQ is an educational demonstration. It cannot account for medical history, examinations, lab results, or symptom severity beyond a coarse weight, and must never be used for real diagnosis.
- **Synthetic, templated data.** The near-deterministic symptom-to-disease mapping inflates accuracy and limits generalisation; real patients present with overlapping and partial symptom sets.
- **Closed world.** The model can only ever output one of 41 known diseases — it has no notion of "unknown," "healthy," or "out of scope," and will confidently force every input into a known class.
- **Static knowledge base.** Medications, diets, and precautions are fixed reference text, not personalised or current clinical guidance.
- **No calibration or uncertainty bounds.** The displayed confidence is a raw forest probability, not a calibrated likelihood, and can look more certain than it is.

11 Future Work

- **Realistic data.** Train and validate on de-identified clinical datasets with genuine noise, comorbidity, and missingness to obtain meaningful performance estimates.
- **An "uncertain" pathway.** Add abstention or out-of-distribution detection so low-confidence or unfamiliar symptom sets are flagged rather than force-classified.
- **Probability calibration.** Apply isotonic or Platt scaling so the confidence gauge reflects true likelihoods.

- **Richer inputs.** Incorporate symptom duration, severity, age, and history; explore gradient-boosted trees or a neural model for comparison.
- **Explainability.** Surface per-prediction feature attributions (e.g. SHAP) so users see *which* symptoms drove the result.
- **Productionisation.** Containerise the service, add automated tests around the preprocessing keys, and serve behind a production WSGI server.

12 Conclusion

MediQ delivers a complete, working machine-learning product in miniature: eight messy spreadsheets are fused into a clean feature matrix and a structured knowledge base; a Random Forest learns to map symptom vectors to diseases and to express its uncertainty as ranked probabilities; and a polished Flask dashboard turns that into an approachable, transparent experience. The headline 100% test accuracy is best read as confirmation that the pipeline is engineered correctly rather than as evidence of clinical skill — and the report has been deliberate about saying so. The real value of the project lies in the journey it demonstrates: disciplined data engineering, a clearly-reasoned modelling choice, honest evaluation, and a deployment that respects the seriousness of its domain. It is a strong foundation, and the future-work directions show exactly how it would evolve from a convincing demonstration toward something genuinely trustworthy.

A Appendix

A.1 The 41 diseases

Fungal infection · Allergy · GERD · Chronic cholestasis · Drug Reaction · Peptic ulcer disease · AIDS · Diabetes · Gastroenteritis · Bronchial Asthma · Hypertension · Migraine · Cervical spondylosis · Paralysis (brain hemorrhage) · Jaundice · Malaria · Chicken pox · Dengue · Typhoid · Hepatitis A · Hepatitis B · Hepatitis C · Hepatitis D · Hepatitis E · Alcoholic hepatitis · Tuberculosis · Common Cold · Pneumonia · Dimorphic hemorrhoids (piles) · Heart attack · Varicose veins · Hypothyroidism · Hyperthyroidism · Hypoglycemia · Osteoarthritis · Arthritis · (vertigo) Paroxysmal Positional Vertigo · Acne · Urinary tract infection · Psoriasis · Impetigo.

A.2 Reproducing the project

1. Install dependencies: `pip install flask pandas scikit-learn`.
2. Train and generate artifacts: `python train_model.py`.
3. Launch the app: `python app.py` and open `http://127.0.0.1:5001`.

A.3 Artifact inventory

Artifact	Contents
disease_model.pkl	Trained forest, label encoder, feature columns, severity map
symptoms.json	132 symptoms with display labels and severity weights (UI autocomplete)
knowledge_base.json	Per-disease description, diets, medications, precautions, lifestyle, symptoms
model_report.txt	Accuracy and full per-class classification report

End of report — MediQ, an educational data-science demonstration. Not for clinical use.